

pyaino プログラムから NumPy 最小実行パッケージを生成し 単体 EXE 化する手順

つくよみちゃん 360d / 12head / 8layer を用いた確定手順

2026-09-21

目的	1 本の pyaino 推論プログラムから、実行に必要な pyaino 部分とデータだけを抽出し、NumPy 専用の最小パッケージを生成して単体 EXE 化する。
最終出力	D:\Python\pyaino_standalone\dist\Tsukuyomi.exe
最小パッケージ	D:\Python\pyaino_standalone\numpy_package_min
検証環境	Windows 11 / Python 3.12.8 / NumPy 2.2.6 / PyInstaller 6.22.3
基本方針	対象プログラム自身が定める相対配置を尊重し、具体例の設定と汎用縮約機構を分離する。

本レポートの範囲 最終的に採用した直線的な手順のみを記載する。検討途中の別案や一時的な配置・探索方法は記載しない。

1. 概要

本手順の目的は、**pyaino** を使った **1 本の推論プログラム**を起点に、実行に必要なモジュール・クラス・関数だけを抽出し、NumPy で動作する最小構成を生成したうえで、PyInstaller により単一 EXE へまとめることである。

処理は次の 5 段階に分かれる。

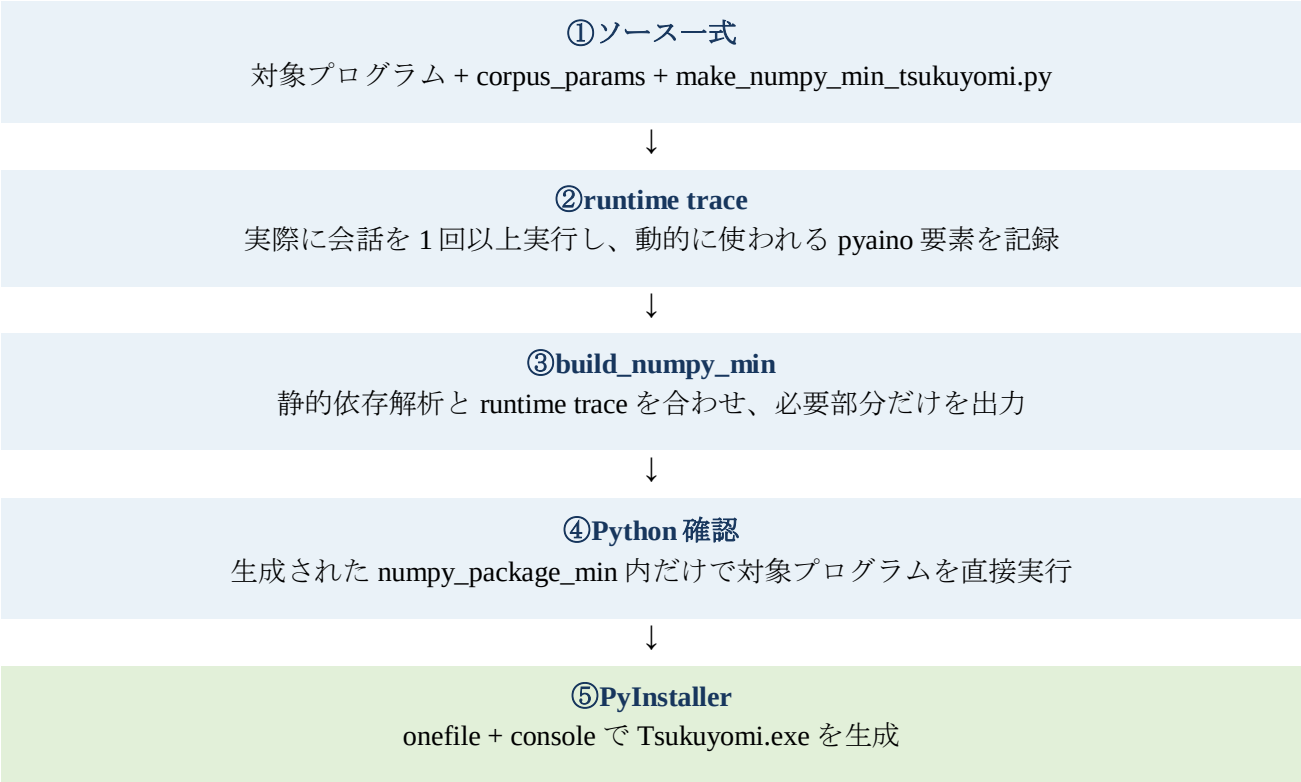


図1 確定した処理フロー

設計上の要点 対象プログラムから見たデータの相対位置は対象プログラム側の規律とし、numpy_min 側が別の場所を探索しない。これにより、実行時の意味とパッケージ生成時の意味が一致する。

2. 最終構成と責務分担

2.1 ソース側の配置

対象プログラム、具体例用ドライバ、実行時データは同じソースツリーに置く。今回の配置は次の通りである。

```
D:\Documents\教育資料\言語モデル\python\
├── generate_GPT_mix_つくよみちゃん会話_supress_print.py
├── make_numpy_min_tsukuyomi.py
├── corpus_params\
│   ├── GPT_tokenizer_tsukuyomi_360d12h8l_20260911.pkl
│   └── GPT_params_tsukuyomi_360d12h8l_20260911_itr137088.pkl
```

汎用機構は pyaino 本体に置く。

```
C:\Python312\Lib\site-packages\pyaino\numpy_min.py
```

ファイル	役割	責任範囲
pyaino/numpy_min.py	汎用の trace・依存解析・最小化・resource コピー	特定モデルの名前やパスを持たない

ファイル	役割	責任範囲
make_numpy_min_tsukuyomi.py	つくよみ固有の TARGET、resource 名、出力先を指定	具体例の構成だけを記述
generate_..._supress_print.py	実際の推論・会話処理	データの相対位置と会話ロジックを所有
corpus_params	Tokenizer / checkpoint	対象プログラムが直接参照する実行時データ

2.2 生成側の配置

```

D:\Python\pyaino_standalone\
├─ PYAINO_TRACE_tsukuyomi.json
├─ numpy_package_min\ ← 再生成可能な最小パッケージ
│   └─ build\          ← PyInstaller 作業領域
│       └─ spec\        ← Tsukuyomi.spec
│           └─ dist\
│               └─ Tsukuyomi.exe ← 最終成果物

```

重要なのは、**numpy_package_min** を再生成可能な中間成果物として扱い、**PyInstaller** の **build / spec / dist** はその外に置くことである。これにより、最小パッケージの再生成と EXE 成果物の管理を分離できる。

3. 対象生成プログラムの最終形

生成プログラムは元の会話生成プログラムを基本とし、EXE 実行時の不要な print 抑止だけを追加した。モデル構造・Tokenizer・checkpoint・会話ロジックは変更しない。

3.1 データファイルは実行プログラム基準

```

from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent

tokenizer.load(
    str(BASE_DIR / "corpus_params" /
        "GPT_tokenizer_tsukuyomi_360d12h8l_20260911.pkl")
)

cf.load_parameters(
    str(BASE_DIR / "corpus_params" /
        "GPT_params_tsukuyomi_360d12h8l_20260911_itr137088.pkl"),
    model
)

```

この書き方により、通常の Python 実行でも、生成後の **numpy_package_min** でも、**PyInstaller onefile** の一時展開先でも、同一の相対配置でデータを取得できる。

3.2 EXE 実行時だけ初期化表示を抑止

```

import sys, os
from contextlib import redirect_stdout

IS_EXE = getattr(sys, "frozen", False)

if IS_EXE:
    _stdout = sys.stdout
    _devnull = open(os.devnull, "w", encoding="utf-8")
    sys.stdout = _devnull

# pyaino import、Tokenizer 読み込み、モデル初期化
# ...

if IS_EXE:
    sys.stdout = _stdout
    _devnull.close()

```

3.3 生成中の内部 print も EXE 時だけ抑止

```
def generate():
    return model.generate(
        seed, len(seed) + 100, stochastic=False,
        skip_ids=skip_ids, end_id=end_id, flush=False)

if IS_EXE:
    with open(os.devnull, "w", encoding="utf-8") as f:
        with redirect_stdout(f):
            gen_data = generate()
else:
    gen_data = generate()
```

したがって、Python で直接実行する開発時には初期化表示や `fix_configuration` を確認でき、配布 EXE では会話表示だけを残せる。

4. make_numpy_min_tsukuyomi.py の役割

具体例用ドライバは、汎用機構に必要な設定を渡す薄いプログラムとした。特定のクラス抽出ロジックや探索ロジックは持たない。

4.1 設定部

```
BASE_DIR = Path(__file__).resolve().parent
TARGET = BASE_DIR / 'generate_GPT_mix_つくよみちゃん会話_supress_print.py'

TOKENIZER_REL = Path('corpus_params') / \
    'GPT_tokenizer_tsukuyomi_360d12h8l_20260911.pkl'
PARAMS_REL = Path('corpus_params') / \
    'GPT_params_tsukuyomi_360d12h8l_20260911_itr137088.pkl'
RESOURCES = [TOKENIZER_REL, PARAMS_REL]

WORK_DIR = Path(r'D:\Python\pyaino_standalone')
TRACE = WORK_DIR / 'PYAINO_TRACE_tsukuyomi.json'
OUTPUT = WORK_DIR / 'numpy_package_min'
FORCE_TRACE = False
```

4.2 ソース一式の存在確認

対象プログラムと 2 つの `resource` が、対象プログラムから見た所定位置に存在することだけを確認する。見つからない場合は別の場所を探索せず、その配置をエラーとして扱う。

4.3 trace と build

```
trace_usage(TARGET, TRACE)

build_numpy_min(
    TARGET,
    output_dir=OUTPUT,
    trace=TRACE,
    resources=RESOURCES,
)
```

既存 `trace` は、正常終了済みで同一 `TARGET` に対するものだけ再利用する。モデル構造や実行経路を意図的に変えた場合は `FORCE_TRACE=True` として取り直す。

5. runtime trace と NumPy 最小パッケージ生成

5.1 trace の実行

`make_numpy_min_tsukuyomi.py` を実行すると、`trace` が必要な場合は対象プログラムが通常どおり起動する。会話が開始したら少なくとも 1 回は応答を生成し、その後空打ちで終了する。これにより、動的経路で実際に呼ばれた `pyaino` 要素を記録できる。

今回の `trace` では、主に次の要素が記録された。

モジュール	記録された主要要素
Activators	Mish, Softmax
BigramLanguageModel	BigramLanguageModel2
Functions	Mean, ScaleDotLinear
Neuron	AttentionUnit, Dropout, Embedding, LinearLayer, MultiHeadSelfAttention, PositionalEmbedding, RMSNormalization, Sequential ほか
common_function	Tokenizer, load_parameters, c2n_dictionary, eval_in_module ほか
nucleus	Function, as_tuple
stems_blocks_heads	FeedForward, LmHead, TransformerBlock

5.2 build 結果

runtime trace と静的依存解析の結果を使い、次の最小パッケージが生成された。

```
D:\Python\pyaino_standalone\numpy_package_min\
├── generate_GPT_mix_つくよみちゃん会話_supress_print.py
├── PACKING_MIN.txt
├── pyaino\
│   └── (実行に必要な最小化済みモジュール)
├── corpus_params\
│   ├── GPT_tokenizer_tsukuyomi_360d12h8l_20260911.pkl
│   └── GPT_params_tsukuyomi_360d12h8l_20260911_itr137088.pkl
```

resource はソース側の相対配置を保ったままコピーされた。

```
D:\Documents\教育資料\言語モデル\python\corpus_params\...
↓
D:\Python\pyaino_standalone\numpy_package_min\corpus_params\...
```

6. numpy_package_min の単体動作確認

生成後は、必ず numpy_package_min 内の対象プログラムを Python で直接実行し、元の pyaino 環境ではなく生成済みローカル pyaino を使って会話できることを確認する。

```
D:\Python\pyaino_standalone\numpy_package_min\
generate_GPT_mix_つくよみちゃん会話_supress_print.py
```

確認時には、起動ログ中の pyaino.Config.py が次の場所を指した。

```
D:\Python\pyaino_standalone\numpy_package_min\pyaino\Config.py
```

Tokenizer と checkpoint も、同じ numpy_package_min 配下の corpus_params から読み込まれた。複数ターンの会話も成立したため、最小パッケージ単体で推論に必要な構成が揃っていることを確認できた。

開発時の表示について Python 直接実行では、Config 初期化、Initialize、fix_configuration 等の内部 print が表示される。これは正常である。print 抑止は IS_EXE=True となる PyInstaller 実行時だけ有効にしている。

7. PyInstaller による単体 EXE 化

7.1 PyInstaller バージョン確認

```
python -m PyInstaller --version
```

今回の成功環境では 6.22.3 を使用した。必要なら次で同版を導入する。

```
python -m pip install pyinstaller==6.22.3
```

7.2 入カー式の確認

```
cd D:\Python\pyaino_standalone\numpy_package_min
dir
dir corpus_params
```

対象プログラム、pyaino、Tokenizer、checkpoint が存在することを確認する。

7.3 PyInstaller 用出力ディレクトリを外側に作成

```
New-Item -ItemType Directory -Force `
"D:\Python\pyaino_standalone\dist", `
"D:\Python\pyaino_standalone\build", `
"D:\Python\pyaino_standalone\spec" | Out-Null
```

7.4 確定したビルドコマンド

```
python -m PyInstaller --clean --noconfirm --onefile --console --name Tsukuyomi `
--exclude-module cupy `
--distpath "D:\Python\pyaino_standalone\dist" `
--workpath "D:\Python\pyaino_standalone\build" `
--specpath "D:\Python\pyaino_standalone\spec" `
--add-data "D:\Python\pyaino_standalone\numpy_package_min\corpus_params\GPT_tokenizer_tsukuyomi_360d12h8l_20260911.pkl:corpus_params" `
--add-data "D:\Python\pyaino_standalone\numpy_package_min\corpus_params\GPT_params_tsukuyomi_360d12h8l_20260911_itr137088.pkl:corpus_params" `
"D:\Python\pyaino_standalone\numpy_package_min\generate_GPT_mix_つくよみちゃん会話_supress_print.py"
```

オプション	意味
--clean	以前の PyInstaller キャッシュを消して再構築する。
--noconfirm	既存出力への確認待ちを省く。
--onefile	配布対象を 1 個の EXE にまとめる。
--console	input() を使う会話用コンソールを持たせる。
--name Tsukuyomi	出力名を Tsukuyomi.exe とする。
--exclude-module cupy	NumPy 専用 EXE とし、CuPy を取り込まない。
--add-data:corpus_params	Tokenizer と checkpoint を EXE 内へ格納し、展開時に corpus_params へ配置する。
--distpath / --workpath / --specpath	EXE 成果物と PyInstaller 作業物を numpy_package_min の外へ分離する。

8. 結果

PyInstaller は次のメッセージで正常終了した。

```
Building EXE from EXE-00.toc completed successfully.
Build complete! The results are available in:
D:\Python\pyaino_standalone\dist
```

最終 EXE は次に生成された。

```
D:\Python\pyaino_standalone\dist\Tsukuyomi.exe
```

エクスプローラーから Tsukuyomi.exe を起動し、会話が正常に開始・継続することを確認した。EXE 実行時は、初期化や attention 内部の print が抑止され、利用者向けの会話表示だけが残る。

確認項目	結果	判定
元の生成プログラム	通常 Python 環境で複数ターン会話	OK
runtime trace	動的利用要素を記録し正常終了	OK

確認項目	結果	判定
numpy_package_min 生成	pyaino 最小構成 + 2 resource を生成	OK
最小パッケージ単体	ローカル pyaino / corpus_params だけで会話	OK
PyInstaller build	onefile build 完了	OK
Tsukuyomi.exe	ダブルクリック起動・会話	OK

9. 考察

今回の確定手順で最も重要なのは、機能を増やすことではなく、責務を分離したことである。

- 対象プログラムは、自分が必要とするデータの相対配置を自分で定義する。
- make_numpy_min_tsukuyomi.py は、具体例の TARGET と resource だけを指定する。
- pyaino.numpy_min は、特定モデルを知らずに trace・依存解析・最小化を担当する。
- numpy_package_min は再生成可能な中間成果物とし、EXE 成果物はその外で管理する。
- Python 開発時の可視性と、EXE 配布時の表示簡潔化を IS_EXE で両立する。

この分離により、つくよみ以外の pyaino 推論プログラムへ適用する際も、原則として具体例ドライバの TARGET と RESOURCES を差し替えるだけで同じ機構を再利用できる。

また、開発 PC では Config 初期化時に CuPy が検出される場合があるが、対象プログラムは set_np('numpy') で NumPy を選び、最終 PyInstaller では --exclude-module cupy を指定している。したがって、今回の配布 EXE は NumPy 側を基準とする構成である。

10. 結論

pyaino を使ったつくよみ会話生成プログラムから、実行に必要な pyaino 部分と実行時データだけを抽出した numpy_package_min を生成し、そのパッケージ単体での Python 実行を確認した。さらに PyInstaller 6.22.3 を用いて onefile / console の Tsukuyomi.exe を生成し、実際に起動して会話できることまで確認した。

これにより、「pyaino の実装を保守しながら、個々の推論プログラムについては NumPy 専用の小さな実行系へ落とし、そのまま単体 EXE へ持っていく」という一連の流れが、実際に再現可能な手順として成立した。

11. 次課題

- 別の pyaino 推論プログラムでも、TARGET / RESOURCES の差し替えだけで同じ手順が成立するかを確認する。
- Python や pyaino を導入していない別 PC で、Tsukuyomi.exe 単体の配布テストを行う。
- 必要に応じて、PyInstaller コマンドを build スクリプト化し、numpy_package_min 生成から EXE 化までを定型化する。
- PACKING_MIN.txt と trace JSON を、再現性確認用のログとして継続保存する。

12. 参考・使用ファイル

- pyaino/numpy_min.py — runtime trace、依存解析、最小化、resource コピーの汎用機構
- make_numpy_min_tsukuyomi.py — つくよみ具体例用ドライバ
- generate_GPT_mix_つくよみちゃん会話_supress_print.py — 元の生成プログラム + EXE 時 print 抑止
- PYAINO_TRACE_tsukuyomi.json — 実行時利用要素の記録

- PACKING_MIN.txt — 最小パッケージ生成結果の manifest
- GPT_tokenizer_tsukuyomi_360d12h8l_20260911.pkl — Tokenizer 辞書
- GPT_params_tsukuyomi_360d12h8l_20260911_itr137088.pkl — checkpoint
- PyInstaller 6.22.3 — 単体 EXE 作成に使用

確定手順の要約 ソース側で対象プログラムと corpus_params を成立させる →
make_numpy_min_tsukuyomi.py で trace/build → numpy_package_min を Python で確認 → PyInstaller で
dist/Tsukuyomi.exe を生成 → EXE を起動確認、の順で進める。